2026

12va EXPO

CIENCIA Y TECNOLOGÍA

PROYECTOS DE DESARROLLO E INNOVACIÓN

DIRECTOR: César Estrebou

INSTITUCIÓN: III-LIDI (UNLP)

SISTEMAS DE NAVEGACIÓN Y ASISTENCIA AUTÓNOMA

Detección de Objetos en Tiempo Real sobre Hardware Embebido

MOTIVACIÓN

- **Transición de Cloud a Edge Computing:** Tradicionalmente, el procesamiento de modelos profundos dependía de la nube. En aplicaciones de movilidad y navegación, el cambio hacia el **Edge Computing** (procesamiento local en el dispositivo) es imprescindible para reducir la latencia, ahorrar ancho de banda y garantizar la seguridad/privacidad de los datos.
- **Desafíos de TinyML/EdgeML:** El reto central reside en adaptar redes neuronales complejas para su ejecución en microcontroladores y placas de recursos reducidos en cómputo y memoria.
- **Impacto Educativo:** Incorporar estas tecnologías en la formación universitaria permite a los alumnos adquirir experiencia práctica directa en la integración de hardware y aprendizaje automático

INFERENCIA EN LA NUBE
ALTA LATENCIA
Datos de video en vivo continuos
Comandos de resultado resultantes
Alta Dependencia de la Red
Recursos Escalables
Posibles Riesgos de Seguridad
INFERENCIA EN EL BORDE
RETARDO CERO
Datos de imagen
Luckfox Pico
Respuesta en Tiempo Real
Uso Reducido del Ancho de Banda
Privacidad Mejorada
Recursos de Computación Limitados

TRABAJO REALIZADO

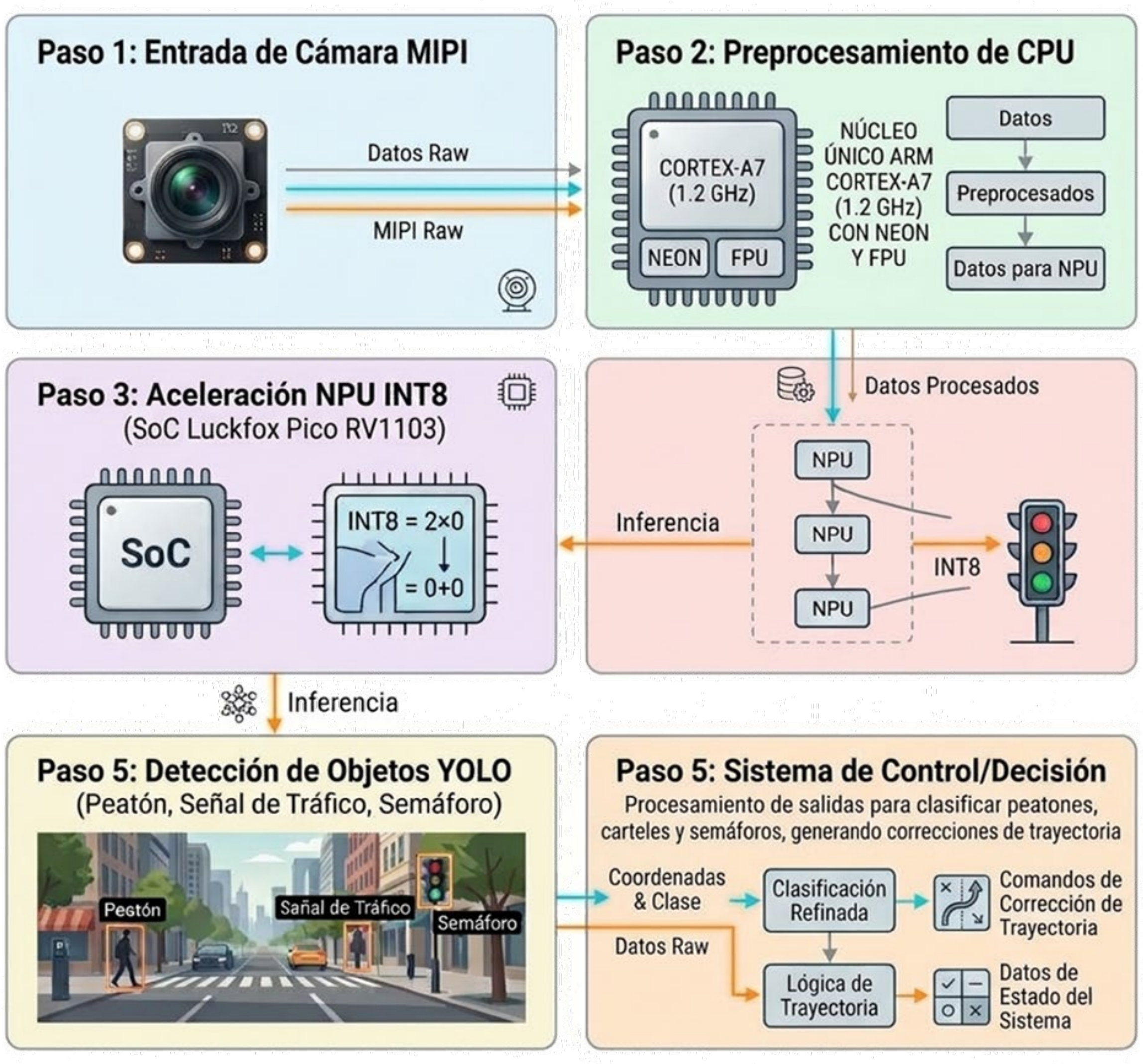
Plataforma Hardware y SO

- **Luckfox Pico Mini (SoC RV1103)**, compuesto por un procesador ARM Cortex-A7 (1.2 GHz) con extensiones NEON y una **NPU** integrada.
- Entorno Linux personalizado compilado mediante **Buildroot**, asegurando un menor tiempo de arranque (*booting*) y maximizando el espacio libre en memoria para los modelos.

Optimización y Despliegue en NPU

- **Conversión y Cuantización:** Conversión del modelo (PyTorch/ONNX) al formato propietario **.rknn** con el **RKNN-Toolkit 2** (`rknn.api`). Se aplicó cuantización **INT8** calibrada con un dataset representativo de 100 imágenes para minimizar la pérdida de precisión.
- **Inferencia y Zero Copy API:** Implementación de un script ejecutable en C utilizando la **Zero Copy API** preasignando buffers compartidos entre el Host y la NPU para eliminar transferencias innecesarias de memoria y reducir la latencia de CPU.

Flujo de Pipeline de Visión Embebida


Paso 1: Entrada de Cámara MIPI
Datos Raw
MIPI Raw
Paso 2: Preprocesamiento de CPU
CORTEX-A7 (1.2 GHz)
NEON
FPU
NÚCLEO ÚNICO ARM CORTEX-A7 (1.2 GHz) CON NEON Y FPU
Datos
Preprocesados
Datos para NPU
Paso 3: Aceleración NPU INT8
(SoC Luckfox Pico RV1103)
SoC
INT8 = 2×0
= 0+0
Datos Procesados
NPU
NPU
NPU
Inferencia
INT8
Inferencia
Paso 5: Detección de Objetos YOLO
(Peatón, Señal de Tráfico, Semáforo)
Pestón
Sañal de Tráfico
Semáforo
Paso 5: Sistema de Control/Decisión
Procesamiento de salidas para clasificar peatones, carteles y semáforos, generando correcciones de trayectoria
Coordenadas & Clase
Datos Raw
Clasificación Refinada
Lógica de Trayectoria
Comandos de Corrección de Trayectoria
Datos de Estado del Sistema

ESPECIFICACIONES DEL SISTEMA

- **SoC:** Rockchip RV1103 (ARM Cortex-A7 @ 1.2 GHz)
- **NPU:** 0.5 TOPS (INT8) / 40.1 GOPS
- **Memoria:** 64 MB DDR2 (Zero-Copy API)
- **Sistema:** Linux Embebido (Buildroot)

Inferencia y Resultados

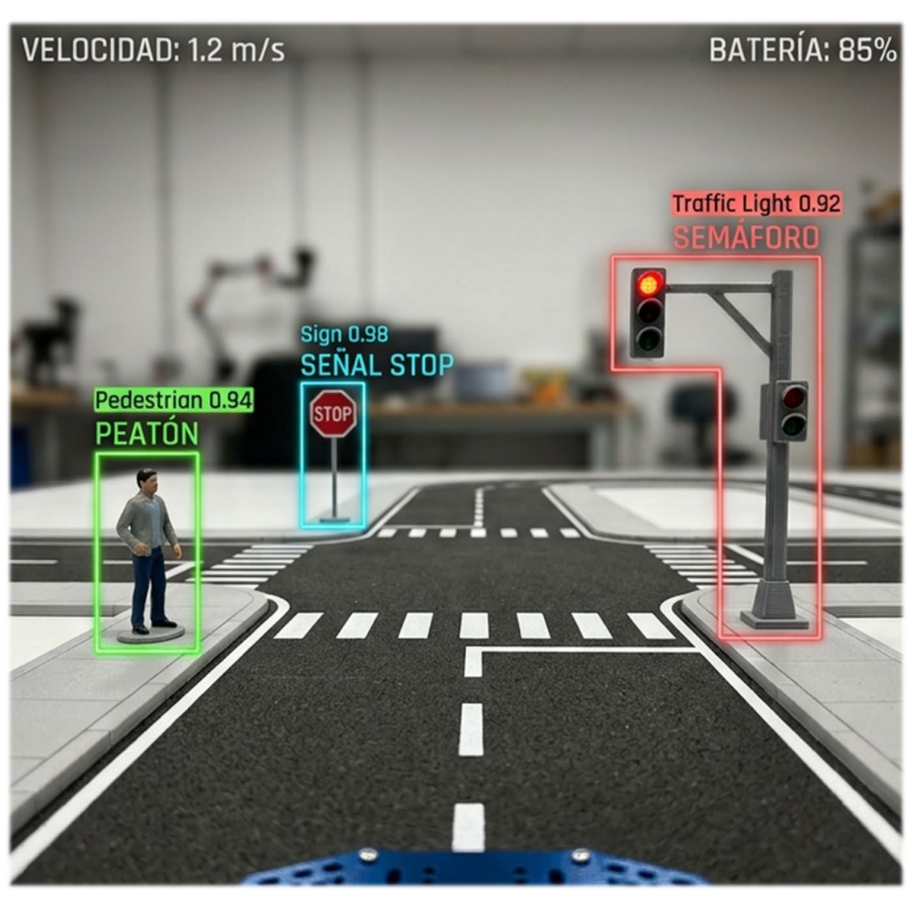
- **Aceleración Masiva por NPU:** En pruebas de laboratorio comparativas, la inferencia ejecutada en la CPU (ARM Cortex-A7 en FP32) requirió en promedio **1.48 segundos** por fotograma, mientras que la ejecución optimizada en la NPU (INT8) completó la tarea en solo **14.9 milisegundos**, representando una aceleración superior al **99%** (>90 veces más rápida).
- **Rendimiento Medido:** Se registraron **40.1 GOPS** (0.0401 TOPS efectivos) durante las corridas de inferencia en la NPU.

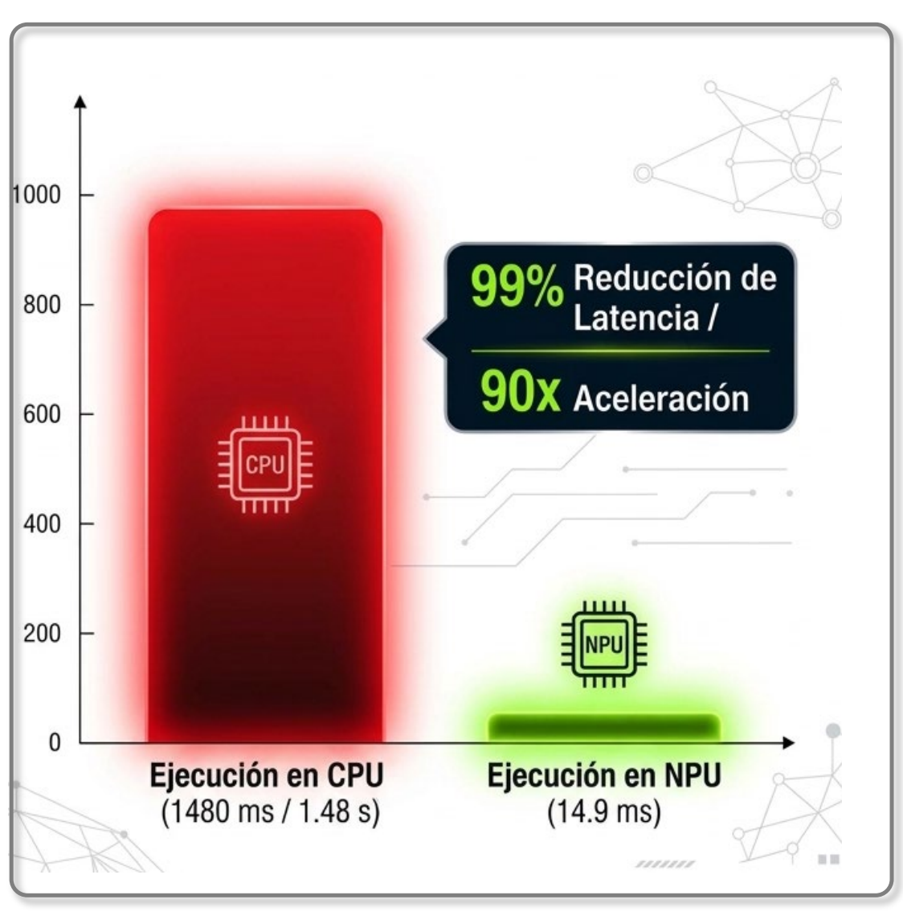
14.9 ms
Latencia NPU

>90 x
Speedup

40.1 GOPS
Performance

INT8
Cuantización





- **Balance Precisión vs. Latencia:** La CPU preserva precisión flotante con latencias inviables para tiempo real. La cuantización a INT8 con NPU da una respuesta fluida para navegación autónoma en el borde.

RESUMEN Y PALABRAS CLAVES

Se desarrolló un prototipo de visión por computadora que detecta en tiempo real peatones, semáforos y carteles en vehículos en miniatura. Se implementó un modelo **YOLO** optimizado en la placa **Luckfox Pico** (SoC RV1103), usando cuantización **INT8** con **aceleración por NPU**. Esto permitió inferencia local de baja latencia, sentando bases para navegación inteligente de bajo costo y experimentación en **TinyML**.

Palabras Clave: *TinyML, Edge Computing, Luckfox Pico, NPU, YOLO, Visión por Computadora, Sistemas Embebidos*

CONCLUSIONES

- Se demostró la viabilidad técnica de ejecutar modelos de detección de objetos en tiempo real (YOLO) sobre hardware embebido de ultra bajo costo.
- El uso coordinado de **Buildroot**, el **RKNN-Toolkit 2** y la **Zero Copy API** resulta indispensable para superar las limitaciones de cómputo y memoria del hardware.
- Se promovió la capacitación de los alumnos en tecnologías avanzadas de **TinyML/EdgeML**, optimización de redes neuronales y trabajo colaborativo en proyectos de I+D.